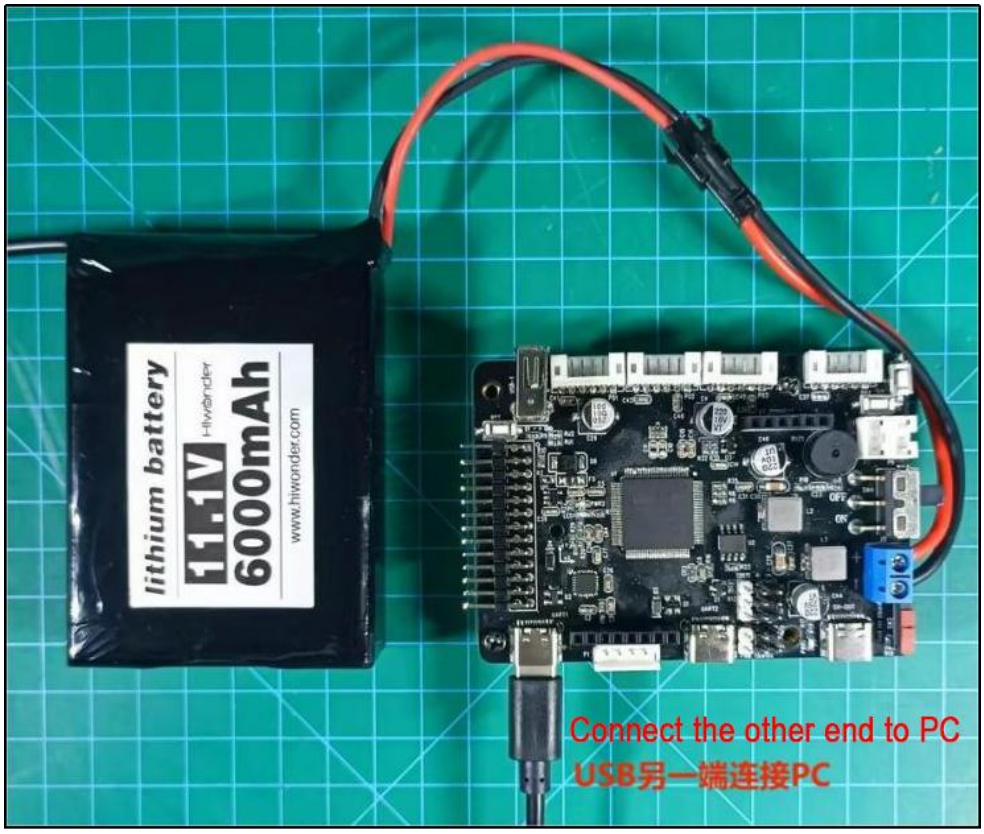
Lesson 2 RRC LED and Buzzer Source Code Analysis

1. Program Logic

This chapter mainly focuses on controlling the LEDs on the STM32 main control board, including turning them on, off, and making them blink.

2. Hardware Connection

The hardware connection diagram is as below:



3. Program Download

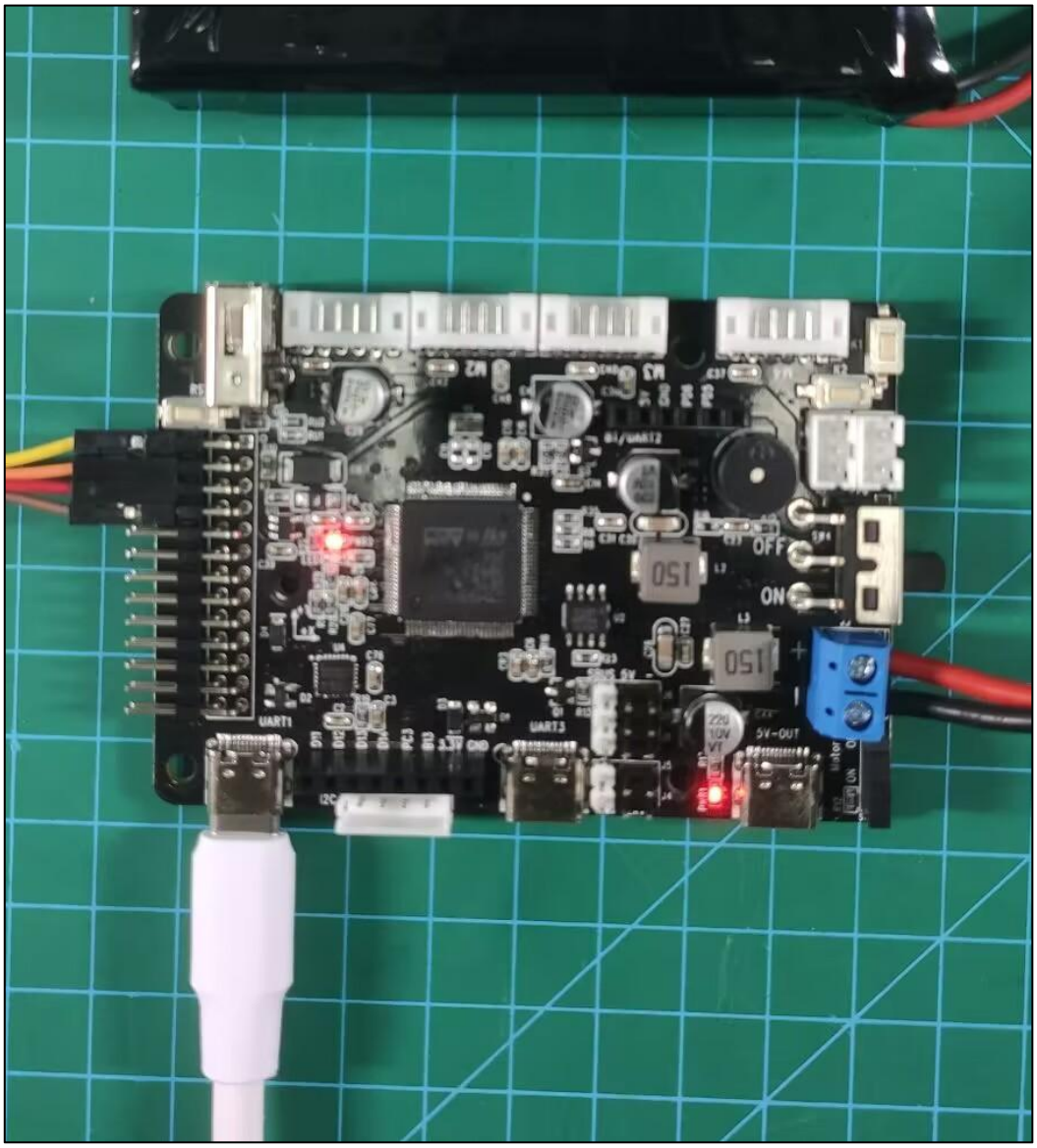
The example for this section is 'rosrobotcontrollerm4_LED_demo'.

For specific instructions on how to download the program, refer to the

document: '3. STM32 Development Basics/Section 3 Project Compilation and Download'.

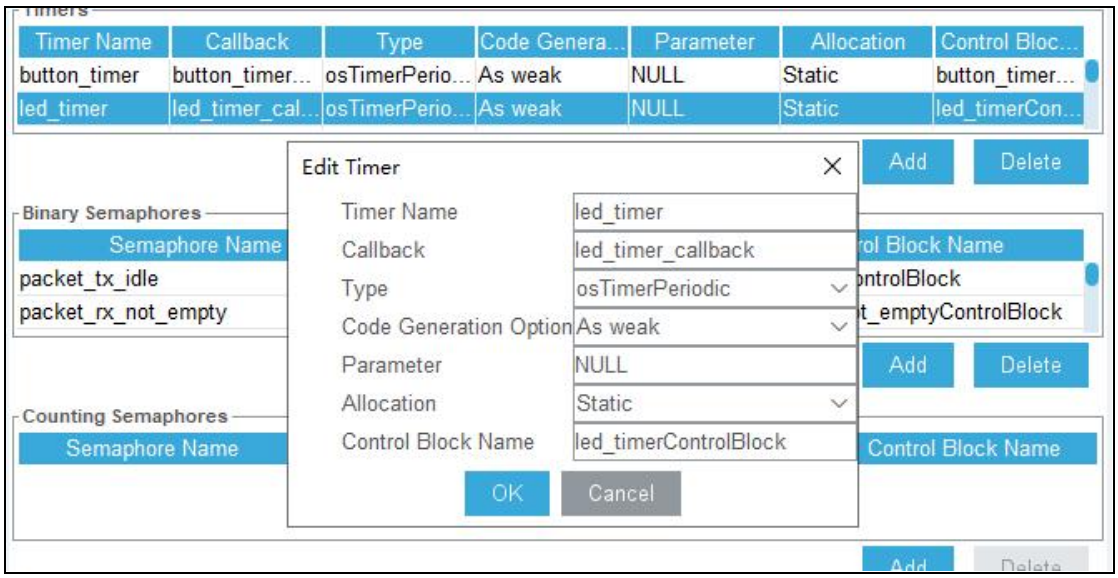
4. Program Outcome

Wait for 3 seconds for the LED to turn on, then another 3 seconds for it to turn off. Afterward, wait 3 seconds for the LED to start blinking (off for 200ms, on for 200ms). After blinking for 3 seconds, the LED will turn off.



5. State Machine Configuration

Use STM32CubeMX to check the information on software timers and state machines. In the 'Pinout & Configuration' section, go to 'FreeRTOS' and open 'Timers and Semaphores'. You will find the details of 'led_task_handler' there.



The current application is in Hiwonder/Peripherals/led.c

6. Source Code Analysis

The source code for this course is located in the same folder as this tutorial document, mainly analyzing the files Hiwonder/System/app.c and Hiwonder/Peripherals/led.c.

6.1 LED Functionality Implementation

```c
#include "cmsis_os2.h"
#include "led.h"
#include "lwmem_porting.h"
#include "global.h"


/* 硬件初始化声明 */
void leds_init(void);   //LED

/* 用户入口函数 */
void app_task_entry(void *argument)
{
  //LED控制句柄
  extern osTimerId_t led_timerHandle;

    /* 硬件初始化 */
    leds_init();    //LED初始化

  //开启LED定时器
  osTimerStart(led_timerHandle, LED_TASK_PERIOD);
  osDelay(300); //延时3秒再亮
  led_on(leds[0]);   //led亮
  osDelay(3000);     //延时3秒再灭
  led_off(leds[0]); //LED灭
  osDelay(3000);
  led_flash(leds[0],200,200,0); //LED闪烁（灭200ms，亮200ms）
  osDelay(3000);
  led_off(leds[0]); //LED灭

  // 循环  : RTOS任务中的循环，必须要有osDelay或者其他系统阻塞函数，否则会导致系统异常
    for(;;) {

        osDelay(300);
        }

    }
```

Above is the App.c file, where app_task_entry() serves as the user's entry function. The main programming flow is as follows:

An externally defined variable led_timerHandle is used as the timer handle.

The leds_init function is called to initialize the LEDs.

Then, the osTimerStart function is used to start the timer, passing LED_TASK_PERIOD as the timer's working interval. This enables the LED control function to be called in the timer interrupt.

First, the LED timer is started, waiting for 3 seconds for the LED to turn on.

Then, waiting for another 3 seconds for the LED to turn off. Subsequently, waiting for 3 seconds to begin LED blinking (off for 200ms, on for 200ms).

After 3 seconds of LED blinking, the light goes off.

The program uses a timer to control the blinking of the LED, and delay functions are used to control the on/off time of the LED and the number of blinks.

6.2 Control LED On, Off and Blinking

```
int led_on(LEDObjectTypeDef *self)
{
    LEDCtrlTypeDef ctrl_structure = {
        .ticks_on = 1,
        .ticks_off = 0,
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

int led_off(LEDObjectTypeDef *self)
{
    LEDCtrlTypeDef ctrl_structure = {
        .ticks_on = 0,
        .ticks_off = 0,
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

int led_flash(LEDObjectTypeDef *self, uint32_t ticks_on, uint32_t ticks_off, uint32_t repeat
{
    LEDCtrlTypeDef ctrl_structure = {
        .ticks_on = ticks_on,
        .ticks_off = ticks_off,
        .repeat = repeat,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}


void led_object_init(LEDObjectTypeDef *self)
{
    self->stage = LED_STAGE_IDLE;
}
```

Above is the LED.c file, which constructs an LED object and provides three methods to control the LED:

The led_on() function is a structure for turning the LED on.

The led_off() function is a structure for turning the LED off.

The led_flash() function is a structure for making the LED blink.

The led_object_init(LEDObjectTypeDef *self) function initializes the LED object. In this function, the state of the state machine is set to LED_STAGE_IDLE.